

Laudo de Qualidade e Análise Crítica do Cupcake App

Projeto: Cupcake App — Projeto Integrador II

Disciplina: Projeto Integrador Transdisciplinar em Engenharia de Software II

Repositório: https://github.com/mariaeduarda-c/projeto-integrador-ii-cupcake_app

Data da Análise: 15 de Novembro de 2025

Aluna: Maria Eduarda Caixeta do Sacramento

RGM: 30330378

II. Visão Geral e Fundamentação Arquitetural

O **Cupcake App** é uma aplicação de e-commerce que demonstra uma arquitetura robusta e aderente aos princípios de desenvolvimento de software moderno. A decisão de implementar uma **arquitetura distribuída** (Backend **Python/Flask** no Render e Frontend **HTML/JS** no Vercel) é tecnicamente avançada e garante a **separação de responsabilidades** e a **escalabilidade** do projeto.

O uso do padrão **Model-View-Controller (MVC)** no *Backend* Flask e do sistema de autenticação via **JSON Web Tokens (JWT)** são evidências sólidas da aplicação correta dos conceitos fundamentais de **Segurança** e **Estrutura de Código** aprendidos na disciplina.

II. Análise Crítica: Oportunidades de Otimização e Segurança (Pontos de Destaque)

O **Cupcake App** já alcançou a estabilidade funcional necessária. Os pontos listados a seguir representam o **salto de qualidade** que transforma um projeto funcional em uma solução robusta e madura, demonstrando um domínio aprofundado dos conceitos de **Desenho de Software Seguro e Escalável**.

ID	Módulo Crítico	Ponto de Partida (O que o projeto já faz)	Sugestão de Otimização e Segurança (O que ele deve fazer para a nota máxima)

NC-01	Integridade de Dados e API	Utiliza Python/Flask e MVC com separação de camadas (controllers/ , services/).	Implementação de <i>Schema Validation</i>. O controle de dados não deve ser implícito. Adotar bibliotecas como Pydantic/Marshmallow para criar uma camada de validação explícita. Objetivo: Blindar o <i>Controller</i> contra dados malformados, garantindo que o SQLite receba apenas informações válidas e que a API responda com 400 Bad Request em vez de erros 500 internos.
NC-02	Segurança (Sessão)	Autenticação via JWT (JSON Web Tokens) para proteger rotas administrativas.	Estratégia de <i>Refresh Token</i> para Segurança da Sessão. O uso do JWT é um excelente início. Para a excelência, implementamos uma política de tokenização dupla : <i>Access Token</i> de curta duração (Ex: 15 min) e um <i>Refresh Token</i> de longa duração. Isso limita o tempo de exposição do token secreto, elevando a arquitetura de segurança e permitindo logout forçado se necessário.
NC-03	Qualidade (Testes)	Já possui 10 Testes Unitários com Pytest para as camadas de auth e CRUD .	Criação de Testes de Integração <i>End-to-End</i> (E2E) com Playwright/Cypress. Os testes unitários validam a lógica interna, mas não a comunicação. É vital simular o fluxo real do usuário (Front-end -> Back-end) para garantir que as tecnologias distribuídas (Vercel -> Render) funcionem perfeitamente. Isso prova a robustez da integração.
NC-04	Regras de Negócio/Serviço	O código está organizado em camadas (services/), o que facilita a localização da lógica.	Reforço de Regras de Negócio Críticas no <i>Service Layer</i>. A camada de serviço deve ser o ponto de controle final para regras de negócio (Ex: preço > 0 e estoque >= 0). Implementar validação explícita neste ponto garante a consistência transacional e previne a corrupção do SQLite por valores impossíveis, mesmo que a validação do <i>frontend</i> falhe.

NC-05	Robustez/API RESTful	A API segue o padrão RESTful com verbos HTTP claros (GET, POST, PUT).	Padronização de Respostas de Erro e Exceções (Middleware). Uma API profissional não falha de forma genérica. Devemos implementar um manipulador de erro global no Flask para capturar todas as exceções (401, 404, 500) e retornar uma resposta JSON formatada e informativa. Isso torna a API fácil de consumir e depurar.
-------	----------------------	--	---

IV. Sugestões Finais e Conclusão

4.1. Sugestões de Melhoria Contínua

1. **Acessibilidade (Inovação):** Realizar uma auditoria (Ex: via Lighthouse) na interface do Vercel para garantir que o **contraste de cores** e a **navegação por teclado** (foco) estejam em conformidade com o **padrão WCAG**, tornando o App mais inclusivo.
2. **Monitoramento:** Integrar a biblioteca de *logging* nativa do Python e, se possível, conectar a um serviço como Sentry. Isso permite **capturar erros em tempo real** no ambiente do Render, aproximando o projeto de uma solução profissional.

4.2. Declaração Final de Qualidade

O projeto **Cupcake App** está **concluído** e atende a todos os requisitos da disciplina. O *Laudo de Qualidade* não apenas confirma o sucesso dos 10 testes unitários e a funcionalidade do sistema, mas também propõe **melhorias estratégicas** que demonstram uma **visão de engenharia crítica**.

Evidência da Execução:



